GeantPy Architecture Roadmap

Vision

GeantPy is a Python-native interface for building, executing, and analyzing Geant4 simulations.

The user should never need to: - Edit YAML files. - Write Geant4 macros. - Interact directly with ROOT files.

Instead, users construct simulations entirely through Python objects. GeantPy translates these objects into a runnable Geant4 workflow, executes the simulation, parses the results, and returns Python-native data structures.

High-Level Architecture

The backend implementation should be completely hidden from the user.

Module Layout

```
geantpy/

    geometry/
    materials/
    particles/
    fields/
    detectors/
    sources/

    simulation/
    backend/
    io/
    analysis/
    visualization/
    units/
```

Each module owns one responsibility.

Phase 1 - Public API

Goal: Design a clean, intuitive interface for constructing simulations.

Example:

```
import geantpy as gp

world = gp.World()

target = gp.Cylinder(
    material="G4_Ir",
    radius=5*gp.mm,
    height=20*gp.mm,
)

beam = gp.Beam(
    particle="proton",
    energy=26*gp.GeV,
)

world.add(target)

sim = gp.Simulation(
    world=world,
    beam=beam,
)

results = sim.run(events=100000)
```

Action items:

- Implement immutable simulation objects.
- Provide SI and Geant4 units.
- Validate parameters during construction.
- Keep the API independent of Geant4 internals.

Phase 2 - Scene Graph

Goal: Represent the complete simulation as Python objects.

Example:

```
World
 ├── Geometry
 ├── Materials
 ├── Particle Source
 ├── Magnetic Fields
 ├── Sensitive Detectors
 └── Physics Configuration
```

Responsibilities:

- Store simulation state.
- Perform validation.
- Serialize into backend configuration.

The scene graph must not execute simulations.

Phase 3 - Backend Interface

Goal: Define a backend abstraction.

```
class Backend:
    def run(simulation):
        ...
```

Planned implementations:

- MacroBackend
- NativeBackend (future)
- RemoteBackend (future)

The Simulation object communicates only with this interface.

Phase 4 - Macro Backend

Initial implementation.

Pipeline:

```
Simulation
      |
      ▼
Generate Geant4 Macro
      |
      ▼
Launch Geant4 Executable
      |
      ▼
Wait for Completion
      |
      ▼
Collect Outputs
```

Responsibilities:

- Generate macro files.
- Manage temporary working directories.
- Execute Geant4.
- Capture stdout/stderr.
- Report execution failures.

This layer should never interpret physics.

Phase 5 - ROOT Parser

Goal: Convert Geant4 outputs into Python objects.

Pipeline:

```
ROOT
    |
    ▼
Parser
    |
    ▼
SimulationResults
```

Responsibilities:

- Read ROOT files.
- Extract tracks.
- Extract hits.
- Extract vertices.
- Extract event summaries.
- Convert data into NumPy-friendly structures.

ROOT remains an implementation detail.

Phase 6 - Results API

The user receives Python objects.

Example:

```
results.tracks
results.hits
results.events
results.energy_deposition
```

Possible future integrations:

- NumPy
- Pandas
- Awkward Array
- PyArrow

Phase 7 - Analysis

Provide lightweight utilities.

Examples:

- Histograms
- Energy spectra
- Angular distributions
- Track filtering
- Event statistics

Heavy scientific workflows belong outside GeantPy.

Design Principles

1. Python First

The public API should feel like NumPy or PyTorch, not Geant4.

2. Backend Agnostic

Simulation objects should not know whether execution occurs through macros, native bindings, or remote infrastructure.

3. Modular

Every package should have one responsibility.

4. Explicit

Construction, execution, and analysis remain separate concerns.

5. Extensible

New geometries, particle sources, detectors, and execution backends should be added without modifying existing interfaces.

Relationship with scientific workflows

GeantPy stops at simulation execution.

```
Python
    ↓
Simulation
    ↓
Geant4
    ↓
Results
```

Projects builds on top of GeantPy.

```
Simulation Results
        │
        ├── Parameter sweeps
        └── Dataset generation
```

This separation keeps GeantPy a general-purpose Geant4 library while allowing projects to focus on higher-level scientific workflows.